A Star Algorithm

AlgoExpert · Very Hard · Famous Algorithms

The problem

- Shortest 4-directional path on a grid (0 free, 1 obstacle) from start to end using Manhattan-distance heuristic.
- Returns the list of [row, col] positions, or [] if unreachable.

Intuition

- Like Dijkstra, but the heap is ordered by distance plus a Manhattan heuristic toward the goal.

Brute force vs optimal

- BFS explores uniformly → many cells
- Dijkstra (h = 0)
- A* with admissible heuristic → far fewer expansions

Approach

- 1. Open set ordered by f equals g plus h
- 2. Expand the best node
- 3. Update neighbours' g
- 4. Rebuild the path when reaching the end

Worked example

```
f = g + h

g: steps from start; h: Manhattan distance to end
```

Priority = cost so far + heuristic

Worked example

```
expand lowest f; skip walls (1)

update g if a shorter path is found
```

Expand the most promising cell

Worked example

```
reach end → follow 'from' pointers back

return the path; unreachable → []
```

Rebuild the path

Java solution (1/2)

```java
public class AStarAlgorithm {
    public static int[][] aStarAlgorithm(int startRow, int startCol, int endRow, int endCol, int[][] graph) {
        int R = graph.length, C = graph[0].length;
        int[][] g = new int[R][C];
        for (int[] row : g) Arrays.fill(row, Integer.MAX_VALUE);
        int[][] from = new int[R * C][];
        PriorityQueue<int[]> open = new PriorityQueue<>(Comparator.comparingInt(x -> x[2]));
        g[startRow][startCol] = 0;
        open.add(new int[]{startRow, startCol, h(startRow, startCol, endRow, endCol)});
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!open.isEmpty()) {
            int[] cur = open.poll();
            int r = cur[0], c = cur[1];
            if (r == endRow && c == endCol) {
                LinkedList<int[]> path = new LinkedList<>();
                for (int[] p = {r, c}; p != null; p = from[p[0] * C + p[1]]) path.addFirst(p);
                return path.toArray(new int[0][]);
            }
            if (cur[2] > g[r][c] + h(r, c, endRow, endCol)) continue;
            for (int[] d : dirs) {
                int nr = r + d[0], nc = c + d[1];
                if (nr < 0 || nc < 0 || nr >= R || nc >= C || graph[nr][nc] == 1) continue;
```

Java solution (2/2)

```
                if (g[r][c] + 1 < g[nr][nc]) {
                    g[nr][nc] = g[r][c] + 1;
                    from[nr * C + nc] = new int[]{r, c};
                    open.add(new int[]{nr, nc, g[nr][nc] + h(nr, nc, endRow, endCol)});
                }
            }
        }
        return new int[][]{};
    }

    private static int h(int r, int c, int er, int ec) { return Math.abs(r - er) + Math.abs(c - ec); }
}
```

Complexity

- Time: O(w*h log(w*h))
- Space: O(w*h)

Edge cases & pitfalls

- Heuristic must be admissible
- Skip stale entries
- Unreachable → []

Interview tips & follow-ups

- Compare with Dijkstra

Recap

- Like Dijkstra, but the heap is ordered by distance plus a Manhattan heuristic toward the goal.
- Complexity: O(w*h log(w*h))